

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: CSS3, Flexbox, Grid y Diseño Responsivo

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

De estilizar elementos a organizar la página completa

En la sesión anterior vimos **selectores**, el **modelo de caja** (margin, border, padding, content) y los valores básicos de `display` (`block` , `inline` , `inline-block`) que usa cada etiqueta HTML por defecto. Con eso ya sabemos **estilizar elementos individuales**: colores, tamaños, tipografía, espaciado.

Pero saber decorar una caja no resuelve un problema distinto: **¿cómo se organizan varias cajas juntas para formar el layout completo de una página?** Un encabezado arriba, un menú a un lado, contenido al centro, un pie abajo — y que todo eso se acomode bien tanto en un monitor grande como en un celular. Hoy resolvemos exactamente eso: **Flexbox**, **Grid** y **diseño responsivo**.

CONTRA EJEMPLO

Antes de Flexbox y Grid: el layout con `float`

Durante años, la única forma de acomodar cajas una junto a otra (por ejemplo, un menú lateral junto al contenido) era la propiedad `float`, pensada originalmente para que el texto **rodeara una imagen**, no para construir layouts completos.

```
.sidebar { float: left; width: 25%; }  
.contenido { float: left; width: 75%; }  
/* el contenedor padre no "sabe" que sus hijos flotan:  
   suele necesitar un hack adicional para no colapsar su altura */
```

Usar `float` para layouts trae problemas conocidos: el contenedor padre **colapsa su altura** (no "ve" a sus hijos flotantes), centrar contenido es incómodo, y distribuir el espacio sobrante entre cajas requiere cálculos manuales de porcentajes. No vamos a profundizar en `float` — solo entender que es la razón por la que Flexbox y Grid **fueron creados específicamente para resolver layouts**, algo para lo que `float` nunca estuvo diseñado.

CONCEPTO

Flexbox: contenedor flex y sus items

Flexbox (*Flexible Box Layout*) es un modelo de CSS pensado para acomodar elementos **en una sola dimensión**: una fila o una columna. Se activa poniendo `display: flex` en un elemento contenedor; todos sus hijos directos se convierten automáticamente en **flex items**.

```
.contenedor {  
  display: flex;  
  flex-direction: row; /* row (por defecto) | column */  
}
```

Flexbox trabaja con dos ejes: el **eje principal** (*main axis*), en la dirección que definas con `flex-direction`, y el **eje cruzado** (*cross axis*), perpendicular al principal. Con `flex-direction: row`, el eje principal es horizontal; con `column`, es vertical. Esta distinción importa porque casi todas las propiedades de Flexbox se definen **en relación a uno de estos dos ejes**, no en términos absolutos de "horizontal" o "vertical".

CONCEPTO

Propiedades del contenedor flex

Propiedad	Controla	Para qué sirve
justify-content	Distribución en el eje principal	Centrar, separar o distribuir espacio entre items
align-items	Alineación en el eje cruzado	Alinear items verticalmente (en <code>row</code>) sin usar <code>margin</code>
flex-wrap	Si los items pueden pasar a otra línea	Evitar que se compriman cuando no caben en una sola fila

```
.contenedor {
  display: flex;
  justify-content: center;      /* flex-start | center | space-between | space-around */
  align-items: center;         /* flex-start | center | flex-end | stretch */
  flex-wrap: wrap;             /* nowrap (por defecto) | wrap */
}
```

Con estas tres propiedades ya se resuelve, por ejemplo, **centrar un elemento perfectamente** (horizontal y vertical a la vez) — algo notoriamente incómodo de lograr solo con el modelo de caja clásico.

CONCEPTO

Propiedades de los items flex

Además de las propiedades del contenedor, cada **item** individual puede controlar cómo crece o se encoge respecto a sus hermanos, con tres propiedades que casi siempre se usan juntas a través de su shorthand:

- `flex-grow` : qué tanto crece el item si sobra espacio (0 = no crece).
- `flex-shrink` : qué tanto se encoge el item si falta espacio (0 = no se encoge).
- `flex-basis` : el tamaño "de partida" del item, antes de repartir espacio sobrante.

```
.item-principal { flex: 1; }           /* shorthand: flex-grow: 1; flex-shrink: 1; flex-basis: 0 */  
.item-fijo      { flex: 0 0 200px; } /* no crece, no se encoge, siempre 200px */
```

`flex: 1` es el patrón más común: hace que el item **ocupe todo el espacio disponible**, repartiéndolo en partes iguales con cualquier otro item que también tenga `flex: 1`.

CONCEPTO

Ejemplo completo: una barra de navegación con Flexbox

Retomando el `<nav>` semántico de la sesión de HTML, Flexbox es la herramienta natural para acomodar su logo y sus enlaces en una fila, con espacio distribuido entre ellos:

```
<nav class="navbar">
  <div class="logo">MiSitio</div>
  <ul class="nav-links">
    <li><a href="#inicio">Inicio</a></li>
    <li><a href="#nosotros">Nosotros</a></li>
    <li><a href="#contacto">Contacto</a></li>
  </ul>
</nav>

.navbar {
  display: flex;
  justify-content: space-between; /* logo a la izquierda, links a la derecha */
  align-items: center;           /* todo alineado verticalmente al centro */
  padding: 1rem 2rem;
}

.nav-links {
  display: flex;           /* Flexbox también dentro del <ul> */
  gap: 1.5rem;             /* espacio uniforme entre los <li> */
  list-style: none;
}
```

CONCEPTO

CSS Grid: layouts en dos dimensiones

CSS Grid es un modelo de layout que, a diferencia de Flexbox, trabaja con **filas y columnas a la vez**. Se activa con `display: grid`, y el tamaño de las columnas/filas se define con `grid-template-columns` y `grid-template-rows`.

```
.contenedor {  
  display: grid;  
  grid-template-columns: 200px 1fr 1fr; /* una columna fija + dos flexibles */  
  grid-template-rows: auto 1fr auto;  
  gap: 1rem; /* espacio entre celdas, filas y columnas */  
}
```

La unidad `fr` ("fracción") reparte el espacio disponible entre columnas o filas de forma proporcional — `1fr 1fr` crea dos columnas de igual ancho. `gap` reemplaza la necesidad de simular espaciado con `margin` en cada celda individual.

CONCEPTO

grid-template-areas : layouts con nombre de zona

Grid permite nombrar visualmente las zonas del layout y luego asignar cada elemento a su zona por nombre, conectando directamente con las etiquetas semánticas (<header> , <aside> , <main> , <footer>) ya vistas en HTML.

```
.pagina {  
  display: grid;  
  grid-template-columns: 220px 1fr;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
}  
header { grid-area: header; }  
aside { grid-area: sidebar; }  
main { grid-area: main; }  
footer { grid-area: footer; }
```

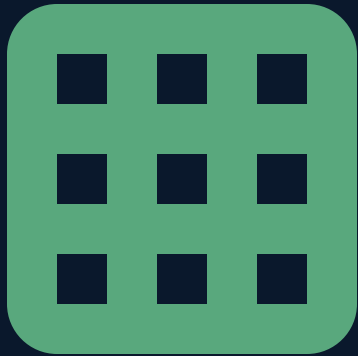
El "mapa" visual en `grid-template-areas` hace que el layout se pueda **leer directamente en el CSS**, sin tener que imaginar mentalmente dónde cae cada fila y columna.

TENSIÓN

¿Flexbox o Grid? No es una elección excluyente

	Flexbox	Grid
Dimensión	Una (fila o columna)	Dos (filas y columnas a la vez)
Ideal para	Barra de navegación, lista de botones, tarjetas en fila	Estructura general de la página, galerías con zonas fijas
Piensa en...	El contenido, que define su propio tamaño	La cuadrícula, que define dónde va cada contenido

En la práctica, **se combinan**: es muy común usar Grid para el esqueleto general de la página (header/sidebar/main/footer) y Flexbox *dentro* de cada una de esas zonas (por ejemplo, dentro del <nav> del header, como en el ejemplo de la barra de navegación).



CONCEPTO

Diseño responsivo y el meta tag viewport

Un **diseño responsivo** es aquel que se adapta a distintos tamaños de pantalla (celular, tablet, monitor) sin necesitar una versión distinta del sitio para cada dispositivo. El primer requisito, obligatorio en toda página HTML, es este meta tag en el `<head>` :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Sin esta línea, los navegadores móviles **simulan una pantalla de escritorio** (típicamente 980px de ancho) y luego reducen todo el sitio para que quepa — el resultado es texto diminuto e ilegible. Con `width=device-width` , el navegador usa el ancho real del dispositivo, y `initial-scale=1.0` evita el zoom inicial no deseado.



CONCEPTO

Media queries: CSS que cambia según el tamaño de pantalla

Una **media query** aplica un bloque de CSS solo cuando se cumple una condición sobre el dispositivo, típicamente su ancho:

```
/* mobile-first: estilos base pensados para pantallas pequeñas */
.contenedor {
  display: grid;
  grid-template-columns: 1fr; /* una sola columna por defecto */
}

@media (min-width: 768px) {
  .contenedor {
    grid-template-columns: 1fr 1fr 1fr; /* tres columnas en pantallas más grandes */
  }
}
```

Mobile-first significa escribir primero el CSS para pantallas pequeñas y usar `min-width` para agregar complejidad en pantallas grandes; **desktop-first** hace lo contrario, partiendo de la versión de escritorio y usando `max-width` para simplificar en pantallas pequeñas. Mobile-first suele preferirse hoy porque la mayoría del tráfico web es móvil.

INTERACCIÓN

De float a Flexbox o Grid

En parejas: este layout de tres bloques (menú, contenido, barra lateral) está hecho con `float`, la técnica antigua que vimos al inicio. Conviértanlo usando Flexbox o Grid — el que les parezca más natural para este caso.

```
<div class="layout">
  <div class="menu">Menú</div>
  <div class="contenido">Contenido</div>
  <div class="lateral">Lateral</div>
</div>

.menu      { float: left; width: 20%; }
.contenido { float: left; width: 60%; }
.lateral   { float: left; width: 20%; }
```

8 minutos · pistas: con Flexbox, piensen en `flex` para los anchos; con Grid, en `grid-template-columns`.

SÍNTESIS

Lo que hay que llevarse de hoy

1. `float` fue la técnica antigua para acomodar cajas, pero no fue diseñada para layouts — de ahí nacen Flexbox y Grid.
2. **Flexbox** (`display: flex`) organiza en **una dimensión**: eje principal y cruzado, `justify-content` / `align-items` / `flex-wrap` en el contenedor, `flex` en los items.
3. **Grid** (`display: grid`) organiza en **dos dimensiones** a la vez, con `grid-template-columns` / `rows` y nombres de zona con `grid-template-areas` .
4. No son excluyentes: Grid para el esqueleto de la página, Flexbox dentro de cada zona.
5. El meta tag `viewport` es obligatorio, y las **media queries** (idealmente mobile-first) adaptan el layout a cada tamaño de pantalla.



CIERRE

Antes de la próxima sesión

- Practicar armando un layout de página completa (header/sidebar/main/footer) combinando Grid para la estructura y Flexbox dentro del `<nav>` .
- Agregar al menos una media query que cambie el layout de una a varias columnas según el ancho de pantalla.

Próxima sesión: JavaScript — dar vida e interactividad a lo que hemos construido en HTML y CSS.

